

Préparation au Concours National Commun — Informatique MP

Corrigé détaillé

Filière MP — Durée : 4 heures

Finalité dans les chaînes de blocs : le protocole Casper
d'après V. Buterin et V. Griffith, *Casper the Friendly Finality Gadget*,
Ethereum Foundation, arXiv:1710.09437v4 (2019)

Auteur : **Pr Agrégé El Hadiq Zouhair**cpgeacademy.org — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun ; il s'en inspire uniquement par la forme et l'esprit.

Pour aller plus loin. Les élèves peuvent traiter une **nouvelle version** de ce problème dotée d'un **autocorrecteur des réponses**, et suivre un **cours complet**, sur cpgeacademy.org.

Note. L'intégralité du code Python de ce corrigé a été exécutée et validée ; les fonctions de la partie IV et la programmation dynamique de la partie V ont en outre été confrontées à une recherche exhaustive sur plusieurs milliers de jeux d'essai aléatoires.

Partie I — Points de contrôle, dépôts et fuite d'inactivité

Question 1. Les deux fonctions se réduisent à une opération arithmétique.

```

1 def est_checkpoint(n):
2     return n % 100 == 0
3
4 def hauteur_bloc(n):
5     return n // 100

```

Chacune effectue une unique division euclidienne : la complexité est $O(1)$ (en supposant, comme d'usage, le coût des opérations arithmétiques sur les entiers manipulés constant).

Question 2. On remonte les liens de parenté jusqu'à la racine.

```

1 def hauteur(parent, c):
2     h = 0
3     x = c
4     while parent[x] != -1:
5         x = parent[x]
6         h = h + 1
7     return h

```

Terminaison. Posons $V = x$ (la valeur courante de la variable x). C'est un entier naturel. À chaque tour de boucle, x est remplacé par $\text{parent}[x]$; or l'hypothèse $0 \leq \text{parent}[c] < c$ garantit $\text{parent}[x] < x$. Le variant V est donc un entier naturel *strictement décroissant* : la boucle effectue un nombre fini de tours. $\square$

Complexité. À chaque tour, la hauteur de x diminue exactement de 1 ; partant de $h(c)$, la boucle effectue exactement $h(c)$ tours, chacun en $O(1)$. La complexité est donc $\Theta(h(c))$, soit $O(n)$ dans le pire des cas (arbre réduit à un chemin).

Question 3. L'hypothèse $\text{parent}[c] < c$ signifie que les nœuds sont numérotés dans un ordre compatible avec la relation de descendance : le père est toujours *déjà* traité.

```

1 def toutes_hauteurs(parent):
2     n = len(parent)
3     h = [0] * n
4     for c in range(n):
5         if parent[c] == -1:
6             h[c] = 0
7         else:
8             h[c] = h[parent[c]] + 1
9     return h

```

Invariant. Au début de l'itération d'indice c , pour tout $c' < c$ on a $h[c'] = h(c')$.

Initialisation. Pour $c = 0$ l'ensemble des indices $c' < 0$ est vide : l'invariant est trivialement vrai.

Hérédité. Supposons l'invariant vrai au début de l'itération c . Si $c = 0$, alors c est la racine et $h(0) = 0$: l'affectation est correcte. Sinon, $p = \text{parent}[c]$ vérifie $0 \leq p < c$, donc par hypothèse d'induction $h[p] = h(p)$; comme le chemin de c à la racine est le chemin de p à la racine augmenté de l'arête $c \rightarrow p$, on a $h(c) = h(p) + 1$, ce qu'affecte l'algorithme. L'invariant est vrai au début de l'itération $c + 1$. $\square$

Conclusion. Après la dernière itération, $h[c'] = h(c')$ pour tout $c' < n$.

C'est *exactement ici* que l'hypothèse $\text{parent}[c] < c$ est indispensable : sans elle, $h[\text{parent}[c]]$ pourrait ne pas encore avoir été calculée et vaudrait la valeur d'initialisation 0, produisant un résultat faux.

Complexité. $\Theta(n)$ en temps et $\Theta(n)$ en espace. À comparer aux $O(n \cdot h)$ qu'aurait coûté n appels à hauteur.

Question 4. Avec $\text{parent} = [-1, 0, 1, 2, 1, 4, 0, 6]$:

c	0	1	2	3	4	5	6	7
$\text{parent}[c]$	-1	0	1	2	1	4	0	6
$h(c)$	0	1	2	3	2	3	1	2

On retrouve bien $h = [0, 1, 2, 3, 2, 3, 1, 2]$.

Question 5. Le point délicat est d'éviter tout flottant.

```

1 def poids(depots, E):
2     s = 0
3     for v in E:
4         s = s + depots[v]
5     return s
6
7 def est_supermajorite(depots, E, T):
8     return 3 * poids(depots, E) >= 2 * T

```

Justification. Comme $3 > 0$, on a l'équivalence $w(E) \geq \frac{2}{3}T \iff 3w(E) \geq 2T$. Les deux membres de droite sont des *entiers* : le test est exact, alors qu'un test $w \geq 2T/3$ en flottants pourrait renvoyer un résultat erroné par arrondi (typiquement lorsque $2T$ n'est pas divisible par 3, ou pour des dépôts de l'ordre de 10^{18} dépassant la précision des *doubles*). Complexité : $O(|E|)$.

Question 6. $T = 30 + 30 + 20 + 20 = 100$, donc $2T = 200$.

- $\{a, b\}$: $w = 60$, $3w = 180 < 200 \Rightarrow$ **pas** une supermajorité.
- $\{a, b, c\}$: $w = 80$, $3w = 240 \geq 200 \Rightarrow$ supermajorité.
- $\{a, c, d\}$: $w = 70$, $3w = 210 \geq 200 \Rightarrow$ supermajorité.

Question 7. (Lemme d'intersection.) Soient E_1, E_2 deux supermajorités.

Le dépôt étant additif sur les ensembles disjoints, la formule d'inclusion-exclusion donne

$$w(E_1 \cup E_2) = w(E_1) + w(E_2) - w(E_1 \cap E_2).$$

Or $E_1 \cup E_2$ est inclus dans l'ensemble de tous les validateurs et les dépôts sont strictement positifs, donc $w(E_1 \cup E_2) \leq T$. Il vient

$$w(E_1 \cap E_2) = w(E_1) + w(E_2) - w(E_1 \cup E_2) \geq \frac{2}{3}T + \frac{2}{3}T - T = \frac{T}{3}.$$

□

Interprétation. Deux supermajorités quelconques se recoupent toujours sur au moins un tiers du dépôt total. C'est ce tiers qui sera détruit en cas d'attaque : toute la sûreté de Casper repose sur ce calcul élémentaire.

Question 8. À chaque époque le dépôt est multiplié par $1 - p$, donc, par récurrence immédiate,

$$D_k = D(1 - p)^k.$$

La perte subie à l'époque k vaut $p D_k$. Comme $0 < 1 - p < 1$, la série géométrique converge et

$$\sum_{k=0}^{+\infty} p D_k = p D \sum_{k=0}^{+\infty} (1 - p)^k = \frac{p D}{1 - (1 - p)} = D.$$

Interprétation. La somme des pertes cumulées converge exactement vers le dépôt initial : la fuite d'inactivité finit par confisquer la *totalité* du dépôt d'un validateur définitivement inactif, mais jamais en temps fini (il en reste toujours une fraction $(1 - p)^k > 0$). Le mécanisme est donc « doux » : il ne punit pas brutalement une absence passagère, mais élimine asymptotiquement les validateurs absents, jusqu'à ce que les validateurs restants redeviennent une supermajorité.

Question 9. Comme $0 < 1 - p < 1$, on a $(1 - p)^k \xrightarrow[k \rightarrow +\infty]{} 0$, donc il existe k tel que $D(1 - p)^k \leq S$: l'ensemble considéré est une partie non vide de $\mathbb{N}$, il admet un plus petit élément. Pour $k \in \mathbb{N}$:

$$D_k \leq S \iff (1 - p)^k \leq \frac{S}{D} \iff k \ln(1 - p) \leq \ln\left(\frac{S}{D}\right) \iff k \geq \frac{\ln(S/D)}{\ln(1 - p)},$$

la dernière équivalence provenant de $\ln(1 - p) < 0$ (le sens de l'inégalité s'inverse). Comme $0 < S < D$, le quotient $x = \ln(S/D)/\ln(1 - p)$ est strictement positif. L'ensemble des entiers $k \geq x$ a pour plus petit élément $\lceil x \rceil$, d'où $k^* = \lceil \ln(S/D)/\ln(1 - p) \rceil$. □

Question 10.

```

1 def nb_epoques(D, p, S):
2     k = 0
3     d = D
4     while d > S:
5         d = d * (1 - p)
6         k = k + 1
7     return k

```

Terminaison. D'après la question 9, l'entier k^* existe. Posons $V = k^* - k$. Tant que la boucle continue on a $d = D_k > S$, donc $k < k^*$ (par minimalité de k^* et décroissance de $k \mapsto D_k$), donc $V \geq 1 > 0$; et V décroît strictement de 1 à chaque tour. C'est un variant entier positif : la boucle termine, et elle s'arrête précisément lorsque $k = k^*$. □

Complexité. Le nombre de tours est $k^* = \lceil \ln(D/S)/\ln(1 - p) \rceil$. À p fixé, $1/\ln(1 - p)$ est une constante, donc $k^* = O(\log(D/S))$, chaque tour coûtant $O(1)$.

Question 11. Pour $D = 1000$, $p = 0,05$, $S = 500$: $x = \frac{\ln(0,5)}{\ln(0,95)} = \frac{-0,693147}{-0,051293} \approx 13,513$, donc $k^* = \lceil 13,513 \rceil = 14$.

Vérification. $D_{13} = 1000 \times 0,95^{13} \approx 513,3 > 500$ et $D_{14} = 1000 \times 0,95^{14} \approx 487,7 \leq 500$. Il faut donc 14 époques pour qu'un validateur inactif perde la moitié de son dépôt.

Partie II — Arbre de points de contrôle et sérialisation des votes

Question 12.

```

1 def est_ancetre(parent, a, b):
2     x = b
3     while x != -1:
4         if x == a:
5             return True
6         x = parent[x]
7     return False

```

Terminaison. Le variant est $V = x + 1 \in \mathbb{N}$: en effet $x \geq -1$ et, à chaque tour, x est remplacé par $\text{parent}[x] < x$. V est un entier naturel strictement décroissant. $\square$

Complexité. La boucle parcourt le chemin de b à la racine, soit au plus $h(b) + 1$ nœuds : la complexité est $O(h(b))$, donc $O(n)$ au pire.

Question 13.

```

1 def conflit(parent, a, b):
2     return not est_ancetre(parent, a, b) and not est_ancetre(parent, b, a)

```

Sur l'arbre A_0 :

- $(2, 4)$: le chemin de 4 à la racine est 4, 1, 0 (ne contient pas 2) et celui de 2 est 2, 1, 0 (ne contient pas 4) : **en conflit**.
- $(3, 5)$: chemins 3, 2, 1, 0 et 5, 4, 1, 0 : **en conflit**.
- $(1, 3)$: le chemin de 3 à la racine est 3, 2, 1, 0 et contient 1 : 1 est ancêtre de 3, donc **pas** de conflit.

Question 14.

```

1 def ancetre_commun(parent, a, b, h):
2     x, y = a, b
3     while h[x] > h[y]: # on remonte le plus profond des deux
4         x = parent[x]
5     while h[y] > h[x]:
6         y = parent[y]
7     while x != y: # meme hauteur : on remonte ensemble
8         x = parent[x]
9         y = parent[y]
10    return x

```

Complexité. Les deux premières boucles effectuent respectivement au plus $h(a)$ et $h(b)$ tours, la troisième au plus $\min(h(a), h(b))$ tours. Le total est $O(h(a) + h(b))$, soit $O(n)$ au pire.

Question 15. Lemme admis (énoncé précis). Si x et y sont deux ancêtres d'un même nœud z et si $h(x) = h(y)$, alors $x = y$. (En effet, les ancêtres de z forment le chemin de z à la racine, sur lequel la hauteur prend chaque valeur de 0 à $h(z)$ exactement une fois.)

Correction des deux premières boucles. Elles préservent l'énoncé « x est un ancêtre de a et y un ancêtre de b », puisqu'on ne fait que remonter des liens de parenté ; elles préservent aussi « tout ancêtre commun de a et b est un ancêtre de x et de y ». Justifions ce second point pour la première boucle : si $h(x) > h(y)$ et si u est un ancêtre commun de a et b , alors $h(u) \leq h(y) < h(x)$, donc $u \neq x$; comme u et x sont tous deux ancêtres de a , u est un ancêtre de x distinct de x , donc un ancêtre de $\text{parent}[x]$. À la sortie, $h(x) = h(y)$.

Invariant de la troisième boucle.

x est un ancêtre de a , y est un ancêtre de b , $h(x) = h(y)$, et tout ancêtre commun de a et b est un ancêtre à la fois de x et de y .

Initialisation. Vrai à l'entrée d'après ce qui précède.

Hérédité. Si $x \neq y$, soit u un ancêtre commun de a et b ; par invariant u est un ancêtre de x et de y . Si l'on avait $u = x$, alors u serait un ancêtre de y de même hauteur que y , donc $u = y$ par le lemme, d'où $x = y$: contradiction. Donc $u \neq x$ et de même $u \neq y$, si bien que u est un ancêtre de $\text{parent}[x]$ et de $\text{parent}[y]$. Les hauteurs restent égales. L'invariant est préservé.

Terminaison. $h(x)$ est un entier naturel qui décroît strictement à chaque tour ; lorsque $h(x) = 0$ on a $x = y = r$ et la boucle s'arrête (au plus tard).

Conclusion. À la sortie $x = y$ est un ancêtre commun de a et b , et tout ancêtre commun de a et b est un ancêtre de x , donc de hauteur $\leq h(x)$. C'est bien le plus proche ancêtre commun. $\square$

Question 16. Notons $\ell = \text{ancetre_commun}(a, b)$.

($\Leftarrow$) Par contraposée. Si a et b ne sont pas en conflit, l'un est ancêtre de l'autre ; disons a ancêtre de b . Alors a est un ancêtre commun de a et b , donc $h(a) \leq h(\ell)$; mais ℓ est un ancêtre de a , donc $h(\ell) \leq h(a)$. D'où $h(\ell) = h(a)$ et, ℓ et a étant deux ancêtres de a de même hauteur, $\ell = a$ par le lemme. Donc $\ell \in \{a, b\}$.

($\Rightarrow$) Par contraposée également. Si $\ell = a$, alors $a = \ell$ est un ancêtre de b , donc a et b ne sont pas en conflit ; de même si $\ell = b$. $\square$

Question 17.

```
1 def vote_valide(parent, depots, vote):
2     v, s, t = vote
3     return (v in depots) and (s != t) and est_ancetre(parent, s, t)
```

Complexité : $O(h(t))$ en moyenne (le test `v in depots` est en $O(1)$ moyen).

Question 18.

```
1 def encode(n):
2     octets = []
3     while True:
4         b = n % 128
5         n = n // 128
6         if n > 0:
7             octets.append(b + 128)
8         else:
9             octets.append(b)
10    return octets
```

$\text{encode}(0) = [0]$, $\text{encode}(127) = [127]$, $\text{encode}(128) = [128, 1]$, $\text{encode}(300) = [172, 2]$.

Détail pour 300 : $300 = 2 \times 128 + 44$, donc $a_0 = 44$, $a_1 = 2$, et le codage est $[44 + 128, 2] = [172, 2]$.

Question 19. Notons $n_0 = n$ et $n_{k+1} = \lfloor n_k/128 \rfloor$ la valeur de la variable `n` après $k+1$ tours. Une récurrence immédiate donne $n_k = \lfloor n/128^k \rfloor$. La boucle s'arrête au premier tour où $n_k = 0$, c'est-à-dire produit exactement

$$\ell = \min\{k \geq 1 : \lfloor n/128^k \rfloor = 0\} = \min\{k \geq 1 : n < 128^k\}$$

`octets`. Si $n = 0$ alors $\ell = 1$. Si $n \geq 1$, $n < 128^k \iff \log_{128} n < k$, et le plus petit entier $k \geq 1$ strictement supérieur à $\log_{128} n$ est $\lfloor \log_{128} n \rfloor + 1$. D'où

$$\ell = \lfloor \log_{128} n \rfloor + 1 = O(\log n).$$

Variant. La suite (n_k) est une suite d'entiers naturels ; tant que la boucle continue on a $n_k \geq 1$ donc $n_{k+1} = \lfloor n_k/128 \rfloor < n_k$. Le variant n_k est un entier naturel strictement décroissant : terminaison. $\square$

Question 20.

```

1 def decode(octets):
2     n = 0
3     p = 1
4     for b in octets:
5         n = n + (b % 128) * p
6         p = p * 128
7     return n

```

Complexité $\Theta(\ell)$ où ℓ est la longueur de la liste.

Question 21. On utilise d'abord la propriété suivante, immédiate d'après le code : pour tout octet b_0 et toute liste L ,

$$\text{decode}([b_0] + L) = (b_0 \bmod 128) + 128 \times \text{decode}(L). \quad (1)$$

Récurrence sur ℓ , le nombre d'octets de $\text{encode}(n)$.

Cas $\ell = 1$. Cela équivaut à $n < 128$ (question 19). Alors $\text{encode}(n) = [n]$ et $\text{decode}([n]) = n \bmod 128 = n$.

Hérédité. Soit $\ell \geq 2$ et supposons la propriété vraie pour tous les entiers dont le codage a $\ell - 1$ octets. Soit n tel que $\text{encode}(n)$ ait ℓ octets, donc $n \geq 128$. Posons $q = \lfloor n/128 \rfloor \geq 1$ et $b = n \bmod 128$. Le premier tour de boucle empile $b + 128$ (car $q > 0$) puis poursuit avec q : par définition de l'algorithme,

$$\text{encode}(n) = [b + 128] + \text{encode}(q),$$

et $\text{encode}(q)$ a $\ell - 1$ octets. En appliquant (1) puis l'hypothèse de récurrence :

$$\text{decode}(\text{encode}(n)) = ((b + 128) \bmod 128) + 128 \times \text{decode}(\text{encode}(q)) = b + 128q = n,$$

la dernière égalité étant la division euclidienne de n par 128. $\square$

Question 22. On a $128^4 = 2^{28} = 268\,435\,456 < 10^9$ et $128^5 = 2^{35} = 34\,359\,738\,368 > 10^9$. D'après la question 19, le codage d'une hauteur $h \leq 10^9$ occupe donc au plus $\lfloor \log_{128} 10^9 \rfloor + 1 = 5$ octets.

Commentaire. Un codage sur 8 octets (entier 64 bits) consommerait toujours 8 octets. En pratique les hauteurs de points de contrôle réelles se comptent en milliers : $1000 < 128^2 = 16\,384$, donc 2 octets suffisent, soit un gain de 75 %. Un vote contenant deux hauteurs, l'économie est de 12 octets par vote ; avec plusieurs dizaines de milliers de validateurs votant à chaque époque, cela représente plusieurs centaines de kilo-octets économisés par époque sur la bande passante du réseau et sur le stockage définitif dans la chaîne. Le prix à payer est un décodage légèrement plus coûteux et la nécessité de connaître la longueur du codage *a posteriori* (d'où le bit de continuation).

Partie III — Justification et finalisation

Question 23. On agrège les dépôts par couple (s, t) , en dédoublonnant au préalable les votes.

```

1 def liens_supermajorite(votes, depots):
2     T = 0
3     for v in depots:
4         T = T + depots[v]
5     poids_lien = {}
6     vus = set()
7     for (v, s, t) in votes:
8         if (v, s, t) not in vus: # un validateur ne compte qu'une fois
9             vus.add((v, s, t))
10            if (s, t) in poids_lien:
11                poids_lien[(s, t)] = poids_lien[(s, t)] + depots[v]
12            else:

```

```

13     poids_lien[(s, t)] = depots[v]
14     L = set()
15     for st in poids_lien:
16         if 3 * poids_lien[st] >= 2 * T:
17             L.add(st)
18     return L

```

Correction. L'ensemble vus garantit qu'un même triplet (ν, s, t) répété n'est comptabilisé qu'une fois ; ainsi $\text{poids_lien}[(s, t)] = w(E_{s \rightarrow t})$ où $E_{s \rightarrow t}$ est l'ensemble des validateurs ayant voté $s \rightarrow t$. Le test final est celui de la question 5.

Complexité. $O(m)$ en moyenne pour la boucle principale (m = nombre de votes, opérations de dictionnaire et d'ensemble en $O(1)$ moyen), plus $O(|L'|)$ pour la boucle finale où $|L'| \leq m$ est le nombre de couples distincts, plus $O(N)$ pour le calcul de T . Total : $O(m + N)$ en moyenne.

Question 24.

```

1 def enfants_de(parent):
2     E = [[] for _ in range(len(parent))]
3     for c in range(len(parent)):
4         if parent[c] != -1:
5             E[parent[c]].append(c)
6     return E

```

Complexité $\Theta(n)$ en temps et en espace (la somme des longueurs des listes vaut $n - 1$).

Remarque. On écrira `[[] for _ in range(n)]` et surtout *pas* `[[]] * n`, qui créerait n références vers une **même** liste.

Question 25.

```

1 def sous_arbre(E, c):
2     r = [c]
3     for f in E[c]:
4         r = r + sous_arbre(E, f)
5     return r

```

Nombre d'appels. Notons $A(c)$ le nombre total d'appels engendrés par `sous_arbre(E, c)` (lui compris). On a $A(c) = 1 + \sum_{f \in E[c]} A(f)$. Par récurrence sur la hauteur du sous-arbre, $A(c)$ est le nombre de descendants de c . Comme chaque nœud $\neq r$ figure dans la liste des fils d'exactly un nœud (son père, unique), le parcours atteint chaque nœud une et une seule fois : $A(0) = n$. $\square$

Coût des concaténations. L'opérateur $+$ sur les listes recopie ses deux opérandes. Sur un arbre réduit à un *chemin* ($\text{parent}[c] = c - 1$), le sous-arbre de c a $n - c$ éléments et l'appel de rang c recopie $n - c$ éléments : le coût total est

$$\sum_{c=0}^{n-1} (n - c) = \frac{n(n+1)}{2} = \Theta(n^2).$$

Variante linéaire. On accumule dans une même liste, sans jamais recopier.

```

1 def sous_arbre_acc(E, c, acc=None):
2     if acc is None:
3         acc = []
4     acc.append(c)
5     for f in E[c]:
6         sous_arbre_acc(E, f, acc)
7     return acc

```

Chaque nœud donne lieu à exactement un `append` en $O(1)$ amorti : la complexité est $\Theta(n)$ en temps. L'espace de pile est $O(h)$ où h est la hauteur de l'arbre.

Question 26. La justification est la *clôture* de $\{r\}$ par les liens de supermajorité : c'est l'ensemble des sommets accessibles depuis r dans le graphe orienté (nœuds, L).

```

1 def justifies(parent, liens, racine=0):
2     succ = {} # liste d'adjacence du graphe des liens
3     for (s, t) in liens:
4         if s in succ:
5             succ[s].append(t)
6         else:
7             succ[s] = [t]
8     J = {racine}
9     pile = [racine]
10    while pile:
11        c = pile.pop()
12        if c in succ:
13            for t in succ[c]:
14                if t not in J:
15                    J.add(t)
16                    pile.append(t)
17    return J

```

Complexité. Construction de succ en $O(|L|)$; chaque sommet est empilé au plus une fois (il est ajouté à J au moment où il est empilé) et chaque arc est examiné au plus une fois. Total : $O(n + |L|)$ en moyenne.

Question 27. Notons J l'ensemble renvoyé et $\mathcal{J}$ l'ensemble des points de contrôle justifiés au sens de la définition.

$J \subseteq \mathcal{J}$ Montrons par récurrence sur l'ordre d'insertion dans J que tout élément inséré est justifié. Le premier inséré est r , justifié par définition. Supposons les k premiers insérés justifiés et soit t le $(k+1)$ -ème : il a été inséré lors du traitement d'un sommet c dépilé, donc déjà inséré, donc justifié par hypothèse de récurrence, et $(c, t) \in L$, c'est-à-dire $c \rightarrow t$. Par définition, t est justifié.

$\mathcal{J} \subseteq J$ Soit $c \in \mathcal{J}$. Par définition, il existe une chaîne $r = b_0 \rightarrow b_1 \rightarrow \dots \rightarrow b_k = c$ de liens de supermajorité (chaque b_i justifiant b_{i+1}). Montrons par récurrence sur k que $c \in J$. Si $k = 0$, $c = r \in J$ par initialisation. Si $k \geq 1$, par hypothèse de récurrence $b_{k-1} \in J$; or tout sommet inséré dans J est empilé, donc dépilé (la boucle ne s'arrête que lorsque la pile est vide, et elle s'arrête car chaque sommet est empilé au plus une fois). Au moment où b_{k-1} est dépilé, l'arc (b_{k-1}, c) est examiné : soit c est déjà dans J , soit il y est ajouté. Dans les deux cas $c \in J$. $\square$

Question 28. On parcourt directement les liens plutôt que le produit $J \times L$.

```

1 def finalises(parent, liens, J, h, racine=0):
2     F = {racine}
3     for (s, t) in liens:
4         if s in J and parent[t] == s and h[t] == h[s] + 1:
5             F.add(s)
6     return F

```

Remarque. Le test $\text{parent}[t] == s$ implique déjà $h[t] == h[s] + 1$; on conserve les deux pour coller littéralement à la définition.

Complexité. $O(|L|)$ en moyenne. (La version naïve parcourant $J \times L$ coûterait $O(|J| \cdot |L|)$.)

Question 29. Soit $c \neq r$ finalisé. Par définition, c est justifié et il existe un lien $c \rightarrow c'$ où c' est un fils direct de c . Or la définition de la justification stipule qu'un point de contrôle x est justifié dès qu'il existe un point justifié y avec $y \rightarrow x$. En prenant $y = c$ (justifié) et $x = c'$, on conclut que c' est justifié. $\square$

Conséquence. Finaliser c , c'est justifier deux points de contrôle **consécutifs** c et c' avec $h(c') = h(c) + 1$: c'est cette « double justification contiguë » qui rend impossible tout emboîtement ultérieur, et donc toute révision de c .

Question 30. Le dépôt total vaut $T = 100$ et $2T = 200$.

Liens. Chacun des couples $(0, 1)$, $(1, 2)$, $(2, 3)$ est voté par $\{a, b, c\}$, de poids $30 + 30 + 20 = 80$; or $3 \times 80 = 240 \geq 200$. Les trois couples sont donc des liens de supermajorité, et ce sont les seuls :

$$L = \{(0, 1), (1, 2), (2, 3)\}.$$

Justifiés. $0 = r$ est justifié ; $0 \rightarrow 1$ donne 1 ; $1 \rightarrow 2$ donne 2 ; $2 \rightarrow 3$ donne 3. Aucun autre nœud n'est cible d'un lien. Donc $J = \{0, 1, 2, 3\}$.

Finalisés. $0 = r$ est finalisé par définition. Pour 1 : il est justifié, le lien $1 \rightarrow 2$ existe et $\text{parent}[2] = 1$ avec $h(2) = h(1) + 1 = 2 \Rightarrow$ finalisé. Pour 2 : justifié, lien $2 \rightarrow 3$, $\text{parent}[3] = 2$, $h(3) = 3 = h(2) + 1 \Rightarrow$ finalisé. Pour 3 : il est justifié mais aucun lien ne part de 3 $\Rightarrow$ non finalisé. Donc

$$F = \{0, 1, 2\}.$$

Question 31.

```

1 def choix_fourche(J, h):
2     meilleur = None
3     for c in J:
4         if meilleur is None or h[c] > h[meilleur]:
5             meilleur = c
6     return meilleur

```

Complexité $\Theta(|J|)$, soit $O(n)$. (L'ensemble J est non vide car il contient toujours la racine.)

Question 32. (a) Soit c justifié. Si $c = r$ la chaîne est réduite à $b_0 = r$. Sinon, la définition fournit un justifié b_{k-1} avec $b_{k-1} \rightarrow c$; en itérant le raisonnement sur b_{k-1} , on construit une suite finie $r = b_0 \rightarrow b_1 \rightarrow \dots \rightarrow b_k = c$ de justifiés (le procédé termine car, un lien $s \rightarrow t$ n'existant que si s est un ancêtre strict de t , la suite des hauteurs $h(b_i)$ est strictement décroissante quand on remonte, donc atteint 0, c'est-à-dire la racine, en un nombre fini d'étapes).

Par définition d'un vote valide, tout lien $s \rightarrow t$ vérifie « s ancêtre strict de t » ; donc chaque b_i est un ancêtre strict de b_{i+1} , et $h(b_i) < h(b_{i+1})$: la suite des hauteurs est strictement croissante. En particulier tout justifié est un descendant de r .

(b) Soit $H = \max\{h(c) : c \in J\}$ (le maximum existe, J étant fini non vide). Supposons deux justifiés $c_1 \neq c_2$ tels que $h(c_1) = h(c_2) = H$. C'est exactement la situation exclue par la propriété (iv) admise. Donc le maximum est atteint en un unique nœud, et la valeur renvoyée par `choix_fourche` ne dépend pas de l'ordre de parcours de l'ensemble J : la règle de choix de fourche est bien définie. $\square$

Commentaire. Cette règle est dite *correcte par construction* : le théorème de vivacité plausible (non démontré ici) assure qu'il est *toujours* possible de finaliser un nouveau point de contrôle au-dessus du justifié de plus grande hauteur, sans qu'aucun validateur honnête n'ait à enfreindre un commandement.

Partie IV — Détection des infractions et sûreté responsable

Question 33. On regroupe les votes par validateur, puis on indexe par hauteur de cible.

```

1 def infraction_I(votes, h):
2     par_val = {}
3     for (v, s, t) in votes:
4         if v in par_val:
5             par_val[v].append((s, t))
6         else:
7             par_val[v] = [(s, t)]
8     for v in par_val:
9         vus = {} # hauteur de cible -> cible déjà vue
10        for (s, t) in par_val[v]:

```

```

11     ht = h[t]
12     if ht in vus and vus[ht] != t:
13         return (v, vus[ht], t)
14     vus[ht] = t
15     return None

```

Correction. Le dictionnaire *vus* mémorise, pour chaque hauteur déjà rencontrée, la cible correspondante. Une infraction à (I) est exactement l'existence de deux votes *distincts* d'un même validateur avec $h(t_1) = h(t_2)$; si les cibles sont égales ($t_1 = t_2$) mais les sources différentes, il s'agit d'une infraction à (I) également au sens strict de l'énoncé du protocole. On a choisi ici de ne signaler que le cas $t_1 \neq t_2$, qui est celui qui met en péril la sûreté (deux branches concurrentes) ; le cas $t_1 = t_2, s_1 \neq s_2$ se détecterait de la même façon en indexant par t .

Complexité. Chaque vote est traité une fois, avec un nombre constant d'opérations de dictionnaire : $O(m)$ en moyenne, $O(m)$ en espace.

Question 34.

```

1 def infraction_II_naif(votes, h):
2     par_val = {}
3     for (v, s, t) in votes:
4         if v in par_val:
5             par_val[v].append((s, t))
6         else:
7             par_val[v] = [(s, t)]
8     for v in par_val:
9         L = par_val[v]
10        for (s1, t1) in L:
11            for (s2, t2) in L:
12                if h[s1] < h[s2] and h[s2] < h[t2] and h[t2] < h[t1]:
13                    return (v, (s1, t1), (s2, t2))
14    return None

```

Complexité. Pour un validateur ν possédant m_ν votes, la double boucle coûte $\Theta(m_\nu^2)$. Le coût total est $\sum_\nu m_\nu^2 \leq (\sum_\nu m_\nu)^2 = m^2$, avec égalité lorsque tous les votes proviennent d'un même validateur : la complexité est $O(m^2)$, atteinte dans le pire des cas.

Question 35. Fixons un validateur ν et notons L la liste de ses votes.

($\Rightarrow$) Supposons que ν enfreigne (II) : il existe (s_1, t_1) et (s_2, t_2) dans L avec $h(s_1) < h(s_2) < h(t_2) < h(t_1)$. Alors (s_1, t_1) appartient à l'ensemble $\{(s, t) \in L : h(s) < h(s_2)\}$, qui est donc non vide, et le maximum des $h(t)$ sur cet ensemble est $\geq h(t_1) > h(t_2)$. Le vote (s_2, t_2) convient.

($\Leftarrow$) Réciproquement, supposons qu'il existe $(s_2, t_2) \in L$ tel que

$$M = \max\{h(t_1) : (s_1, t_1) \in L, h(s_1) < h(s_2)\} > h(t_2).$$

Soit (s_1, t_1) un vote réalisant ce maximum. On a alors $h(s_1) < h(s_2)$ (appartenance à l'ensemble), $h(s_2) < h(t_2)$ (car le vote (s_2, t_2) est *valide* : s_2 est un ancêtre strict de t_2), et $h(t_2) < M = h(t_1)$. Les quatre inégalités $h(s_1) < h(s_2) < h(t_2) < h(t_1)$ sont donc satisfaites, et les deux votes sont distincts (leurs sources n'ont pas la même hauteur). Le commandement (II) est enfreint. $\square$

C'est ici qu'intervient de façon décisive l'hypothèse de validité des votes : sans elle, la condition intermédiaire $h(s_2) < h(t_2)$ devrait être testée explicitement.

Question 36. L'équivalence précédente suggère de trier par $h(s)$ croissant et de maintenir le maximum courant des $h(t)$. Il faut traiter les votes de **même** hauteur de source par *groupes*, car l'inégalité $h(s_1) < h(s_2)$ est stricte.

```

1 def infraction_II(votes, h):
2     par_val = {}
3     for (v, s, t) in votes:
4         if v in par_val:

```

```

5     par_val[v].append((s, t))
6     else:
7         par_val[v] = [(s, t)]
8     for v in par_val:
9         L = sorted(par_val[v], key=lambda st: h[st[0]])
10        n = len(L)
11        M = -1 # max des h(t) sur les votes de source STRICTEMENT plus basse
12        temoin = None # un vote realisant ce maximum
13        i = 0
14        while i < n:
15            j = i # [i, j[ = groupe de meme h(s)
16            while j < n and h[L[j][0]] == h[L[i][0]]:
17                j = j + 1
18            for k in range(i, j): # (s2, t2) candidats
19                if M > h[L[k][1]]:
20                    return (v, temoin, L[k])
21            for k in range(i, j): # on integre le groupe courant
22                if h[L[k][1]] > M:
23                    M = h[L[k][1]]
24                    temoin = L[k]
25            i = j
26    return None

```

Invariant de la boucle principale.

Au début de chaque itération (d'indice i), M est le maximum des $h(t_1)$ sur les votes (s_1, t_1) de L tels que $h(s_1) < h(L[i]_{\text{source}})$, avec la convention $M = -1$ si cet ensemble est vide, et *temoin* est un vote réalisant ce maximum.

Initialisation. Pour $i = 0$, aucun vote n'a de source de hauteur strictement inférieure à la plus petite : l'ensemble est vide et $M = -1$.

Hérédité. La liste étant triée par $h(s)$ croissant, les votes d'indice $< i$ sont exactement ceux dont la hauteur de source est $\leq h(L[i]_{\text{source}})$; la tranche $[i, j[$ regroupe tous ceux de hauteur de source égale. Après l'exécution de la seconde boucle interne, M intègre donc précisément les votes de hauteur de source $\leq h(L[i]_{\text{source}})$, c'est-à-dire strictement inférieure à $h(L[j]_{\text{source}})$: l'invariant est rétabli pour $i \leftarrow j$.

Correction. Grâce à l'invariant, le test $M > h[L[k][1]]$ de la première boucle interne est exactement la condition de la question 35 pour le vote candidat $L[k]$. La fonction renvoie donc un témoin si et seulement si ν enfreint (II).

Terminaison. La quantité $n - i$ est un entier naturel qui décroît strictement à chaque tour, car $j > i$ (le groupe est non vide).

Complexité. Le regroupement par validateur coûte $O(m)$ en moyenne. Pour chaque validateur, le tri coûte $O(m_\nu \log m_\nu)$ et le double parcours $O(m_\nu)$. Comme $\sum_\nu m_\nu \log m_\nu \leq m \log m$, la complexité totale est

$$O(m \log m)$$

contre $O(m^2)$ pour la version naïve. L'espace supplémentaire est $O(m)$.

Vérification expérimentale. Cette fonction et `infracction_II_naif` ont été comparées sur 20 000 jeux de votes valides tirés au hasard : elles concordent toujours, et le témoin renvoyé vérifie bien $h(s_1) < h(s_2) < h(t_2) < h(t_1)$.

Question 37. Sur A_0 , $h = [0, 1, 2, 3, 2, 3, 1, 2]$.

- Votes $(0, 3)$ et $(1, 2)$: $h(0) = 0$, $h(1) = 1$, $h(2) = 2$, $h(3) = 3$, d'où $0 < 1 < 2 < 3$, c'est-à-dire $h(s_1) < h(s_2) < h(t_2) < h(t_1)$ avec $(s_1, t_1) = (0, 3)$ et $(s_2, t_2) = (1, 2)$: le validateur enfreint le **commandement (II)** (vote emboîté).

- Votes $(0, 2)$ et $(0, 4)$: $h(2) = h(4) = 2$ et $2 \neq 4$: deux votes distincts pour une même hauteur de cible, le validateur enfreint le **commandement (I)**. Notons que 2 et 4 sont précisément en conflit (question 13) : ce validateur soutient simultanément deux branches concurrentes.

Question 38. On suppose que l'ensemble $\mathcal{F}$ des validateurs fautifs vérifie $w(\mathcal{F}) < T/3$.

(i) Soient $s_1 \rightarrow t_1$ et $s_2 \rightarrow t_2$ deux liens de supermajorité *distincts* (c'est-à-dire $(s_1, t_1) \neq (s_2, t_2)$) et supposons $h(t_1) = h(t_2)$. Notons E_1 et E_2 les ensembles de validateurs ayant respectivement voté $s_1 \rightarrow t_1$ et $s_2 \rightarrow t_2$: ce sont deux supermajorités. D'après le lemme d'intersection (question 7), $w(E_1 \cap E_2) \geq T/3$. Or tout $\nu \in E_1 \cap E_2$ a publié les deux votes distincts $\langle \nu, s_1, t_1 \rangle$ et $\langle \nu, s_2, t_2 \rangle$ avec $h(t_1) = h(t_2)$: il enfreint le commandement (I), donc $\nu \in \mathcal{F}$. Ainsi $T/3 \leq w(E_1 \cap E_2) \leq w(\mathcal{F}) < T/3$: contradiction. Donc $h(t_1) \neq h(t_2)$. $\square$

(ii) Même raisonnement. Si $h(s_1) < h(s_2) < h(t_2) < h(t_1)$, les deux liens sont nécessairement distincts, et tout $\nu \in E_1 \cap E_2$ a publié deux votes réalisant exactement la configuration interdite par le commandement (II). On obtient de nouveau $w(\mathcal{F}) \geq T/3$, contradiction. $\square$

Question 39. (iii) Supposons deux liens de supermajorité $s_1 \rightarrow t_1$ et $s_2 \rightarrow t_2$ avec $h(t_1) = h(t_2) = n$. S'ils sont distincts, (i) est contredit. Ils sont donc égaux : il existe au plus un lien de supermajorité de hauteur de cible n . $\square$

(iv) Si $n = 0$, le seul point de contrôle de hauteur 0 est la racine. Si $n \geq 1$, soient c_1 et c_2 deux justifiés de hauteur n . Comme $n \geq 1$, aucun n'est la racine, donc il existe des liens $c'_1 \rightarrow c_1$ et $c'_2 \rightarrow c_2$. Leurs cibles ont même hauteur n ; par (iii) ces deux liens sont égaux, donc en particulier $c_1 = c_2$. Il existe donc au plus un point de contrôle justifié de hauteur n . $\square$

Question 40. (Théorème de sûreté responsable.) Soient a_m et b_n deux points de contrôle finalisés en conflit, de fils directs justifiés a_{m+1} et b_{n+1} , avec $h(a_{m+1}) = h(a_m) + 1$ et $h(b_{n+1}) = h(b_n) + 1$. Supposons, comme le permet la symétrie, $h(a_m) < h(b_n)$ (le cas d'égalité est traité à la question suivante), et raisonnons par l'absurde en supposant $w(\mathcal{F}) < T/3$, de sorte que (i)–(iv) s'appliquent.

D'après la question 32(a), il existe une chaîne de justification

$$r = b_{i_0} \rightarrow b_{i_1} \rightarrow \dots \rightarrow b_{i_k} = b_n$$

dont tous les termes sont justifiés et dont les hauteurs sont strictement croissantes. Notons pour alléger $\beta_0 = r$, $\beta_1, \dots, \beta_k = b_n$ ces points de contrôle.

Étape 1. Aucune hauteur $h(\beta_p)$ ne vaut $h(a_m)$ ni $h(a_{m+1})$. En effet, β_p est justifié ; a_m et a_{m+1} le sont aussi (a_m car finalisé, a_{m+1} par la question 29). Si $h(\beta_p) = h(a_m)$, la propriété (iv) impose $\beta_p = a_m$, donc a_m serait un ancêtre de b_n (les β formant une chaîne d'ancêtres de b_n), ce qui contredit le conflit entre a_m et b_n . Même argument pour a_{m+1} , dont a_m est un ancêtre.

Étape 2. Comme $h(\beta_0) = 0 \leq h(a_m)$ (et $h(\beta_0) \neq h(a_{m+1})$) tandis que $h(\beta_k) = h(b_n) > h(a_m)$, l'ensemble $\{p : h(\beta_p) > h(a_{m+1})\}$ est non vide dès que $h(b_n) > h(a_{m+1})$; c'est le cas, car $h(b_n) > h(a_m)$ et $h(b_n) \neq h(a_{m+1})$ par l'étape 1. Soit j le plus petit tel indice. Alors $j \geq 1$ et, par minimalité, $h(\beta_{j-1}) < h(a_{m+1})$; l'étape 1 interdit l'égalité, donc $h(\beta_{j-1}) < h(a_{m+1})$, et même $h(\beta_{j-1}) < h(a_m)$ puisque $h(\beta_{j-1})$ ne peut valoir ni $h(a_m)$ ni $h(a_{m+1})$ et que $h(a_{m+1}) = h(a_m) + 1$.

Étape 3. On dispose donc du lien de supermajorité $\beta_{j-1} \rightarrow \beta_j$ avec

$$h(\beta_{j-1}) < h(a_m) < h(a_{m+1}) < h(\beta_j),$$

et du lien de supermajorité $a_m \rightarrow a_{m+1}$ (qui existe puisque a_m est finalisé). C'est exactement la configuration interdite par la propriété (ii), avec $(s_1, t_1) = (\beta_{j-1}, \beta_j)$ et $(s_2, t_2) = (a_m, a_{m+1})$: contradiction.

Conclusion. L'hypothèse $w(\mathcal{F}) < T/3$ est absurde. Autrement dit, si deux points de contrôle en conflit sont tous deux finalisés, alors

$$w(\mathcal{F}) \geq \frac{T}{3},$$

c'est-à-dire qu'au moins un tiers du dépôt total est détruit, et l'on sait *quels* validateurs punir : c'est la **sûreté responsable** (*accountable safety*). $\square$

Commentaire. Cette propriété est plus forte que la sûreté des algorithmes BFT classiques : non seulement une attaque est impossible tant que moins d'un tiers des participants trichent, mais si elle a lieu, elle est *prouvable* et *sanctionnable* sur la chaîne. C'est ce qui résout le problème du *nothing at stake* propre à la preuve d'enjeu.

Question 41. Si $h(a_m) = h(b_n)$ avec $a_m \neq b_n$ (ils sont en conflit donc distincts), on a deux points de contrôle justifiés distincts de même hauteur, ce qui contredit directement (iv). En remontant à la source, ce sont les deux liens $a'_{m-1} \rightarrow a_m$ et $b'_{n-1} \rightarrow b_n$ dont les cibles ont même hauteur : les validateurs de l'intersection (de poids $\geq T/3$) ont publié deux votes distincts de même hauteur de cible et enfreignent donc le **commandement (I)**.

Question 42. Les tirages sont indépendants, uniformes et avec remise : à chaque tirage, la probabilité que le validateur obtenu soit fautif vaut q (proportion de fautifs), indépendamment des tirages précédents. La variable X compte le rang du premier succès dans une suite d'épreuves de Bernoulli indépendantes de paramètre q : c'est par définition une **loi géométrique** de paramètre q , à valeurs dans $\mathbb{N}^*$. Pour $k \geq 1$:

$$P(X = k) = (1 - q)^{k-1}q, \quad P(X > k) = \sum_{i>k} (1 - q)^{i-1}q = (1 - q)^k, \quad \mathbb{E}[X] = \frac{1}{q}.$$

(Le calcul de $P(X > k)$ s'obtient plus simplement en remarquant que $X > k$ signifie que les k premiers tirages sont des échecs, événements indépendants de probabilité $1 - q$ chacun.)

Question 43. L'événement $\{X = +\infty\}$ (l'algorithme ne s'arrête jamais) est l'intersection décroissante des événements $\{X > k\}$, donc par continuité décroissante de la probabilité

$$P(X = +\infty) = \lim_{k \rightarrow +\infty} P(X > k) = \lim_{k \rightarrow +\infty} (1 - q)^k = 0,$$

puisque $0 < 1 - q < 1$. L'algorithme termine donc **presque sûrement**. $\square$

Question 44.

```

1 import random
2
3 def recherche_fautif(validateurs, est_fautif):
4     N = len(validateurs)
5     n_essais = 0
6     while True:
7         i = random.randrange(N)
8         n_essais = n_essais + 1
9         if est_fautif(validateurs[i]):
10             return (validateurs[i], n_essais)

```

Espérance. Le nombre d'appels à `est_fautif` est exactement X . Pour $q = \frac{1}{3}$, $\mathbb{E}[X] = 1/q = 3$: en moyenne trois vérifications suffisent, quel que soit le nombre N de validateurs. C'est ce qui rend la vérification par échantillonnage praticable pour un client léger.

Pire des cas. Il n'existe *aucune* borne : pour tout k , $P(X > k) = (1 - q)^k > 0$. L'algorithme est de type *Las Vegas* : son résultat est toujours correct, mais son temps d'exécution n'est borné qu'en espérance. On ne peut donc parler que de complexité *en moyenne*, ici $\Theta(1/q)$ appels.

Partie V — Coût minimal d'une attaque (programmation dynamique)

Question 45. Les dépôts étant entiers, $w(E)$ est un entier. Pour un entier w :

$$w \geq \frac{2T}{3} \iff w \geq \left\lceil \frac{2T}{3} \right\rceil,$$

car le plus petit entier supérieur ou égal à un réel x est $\lceil x \rceil$. Donc $C = \lceil 2T/3 \rceil$ convient, et c'est bien le plus petit tel entier (pour $C' < C$, l'entier $w = C'$ vérifierait $w \geq C'$ sans être une supermajorité, dès lors que C' est atteignable). En Python, avec des divisions entières uniquement :

```
1 C = (2 * T + 2) // 3 # ou, de façon équivalente : C = -((-2 * T) // 3)
```

Vérification : $T = 60 \Rightarrow C = 40$; $T = 100 \Rightarrow C = 67$; $T = 101 \Rightarrow C = 68$.

Question 46. Appartenance à NP. Un *certificat* pour l'instance (d, C) est la liste des indices d'un sous-ensemble $S \subseteq \{0, \dots, n-1\}$. Sa taille est $O(n \log n)$ bits, donc polynomiale en la taille de l'entrée. Le *vérificateur* calcule $\sum_{i \in S} d_i$ par $|S| - 1 \leq n - 1$ additions d'entiers de taille au plus celle de l'entrée, puis compare à C : c'est un calcul en temps polynomial. Le problème est donc dans NP. $\square$

Conséquence pour Coût-Attaque. Un algorithme calculant $\mu(d, C)$ en temps polynomial résoudrait SOMME-DE-SOUS-ENSEMBLE en temps polynomial : il suffit de tester si $\mu(d, C) = C$. En effet, $\mu(d, C)$ est la plus petite somme $\geq C$; elle vaut exactement C si et seulement s'il existe un sous-ensemble de somme exactement C . Comme SOMME-DE-SOUS-ENSEMBLE est NP-complet, un tel algorithme entraînerait $P = NP$. Sous l'hypothèse $P \neq NP$, il n'existe donc **pas** d'algorithme polynomial pour COÛT-ATTAQUE.

Question 47. Initialisation. Pour $i = 0$ le seul sous-ensemble de $\emptyset$ est $\emptyset$, de somme 0 :

$$m_0(0) = 0 \quad \text{et} \quad m_0(j) = +\infty \quad \text{pour} \quad 1 \leq j \leq C.$$

Récurrence. Un sous-ensemble $S \subseteq \{0, \dots, i\}$ contient ou non l'indice i . En notant $\pi(x) = \min(C, x)$ l'écrêtage, on obtient pour tout $0 \leq j \leq C$:

$$m_{i+1}(j) = \min \left(m_i(j), \min \{ m_i(j') + d_i : 0 \leq j' \leq C, \pi(j' + d_i) = j \} \right),$$

avec la convention $\min \emptyset = +\infty$. En pratique on implémente cette relation « en avant » : pour chaque j' tel que $m_i(j') < +\infty$, on met à jour l'entrée d'indice $\pi(j' + d_i)$ avec la valeur candidate $m_i(j') + d_i$.

Rôle de l'écrêtage. Sans écrêtage, l'indice j devrait parcourir $\llbracket 0, \sum d_i \rrbracket$, tableau dont la taille dépend des données et non du seuil. L'écrêtage borne la taille à $C + 1$. Il ne fait perdre aucune solution optimale : toutes les sommes $\geq C$ sont regroupées dans la case C , mais on y conserve le *minimum* des valeurs réelles correspondantes, et c'est précisément ce minimum que l'on cherche. Aucune information utile n'est donc perdue.

Remarque. Pour $j < C$ la définition impose $\pi(\sum_{k \in S} d_k) = j$ avec $j < C$, donc $\sum_{k \in S} d_k = j$: ainsi $m_i(j) \in \{j, +\infty\}$. Le tableau se comporte comme un simple test d'atteignabilité pour $j < C$, et seule la case C porte réellement une optimisation.

Question 48. Par définition, $\mu(d, C)$ est le minimum des sommes $\geq C$ sur les sous-ensembles de $\{0, \dots, n-1\}$; ce sont exactement les sous-ensembles dont la somme écrêtée vaut C (pour $C \geq 1$). Donc

$$\mu(d, C) = m_n(C)$$

(avec la convention $\mu = +\infty$ si $m_n(C) = +\infty$; et si $C = 0$, $\mu = 0$).

Question 49.

```
1 def cout_min_attaque(d, C):
2     INF = float('inf')
3     m = [INF] * (C + 1)
4     m[0] = 0
5     for i in range(len(d)):
6         for j in range(C, -1, -1): # parcours DECROISSANT : indispensable
7             if m[j] < INF:
8                 jp = min(C, j + d[i])
```

```

9      if m[j] + d[i] < m[jp]:
10         m[jp] = m[j] + d[i]
11      return m[C]

```

Sens de parcours. Comme $d_i \geq 1$, l'indice cible $j' = \min(C, j + d_i)$ vérifie $j' > j$, sauf lorsque $j = C$ (où $j' = C$, mais l'affectation est alors sans effet puisque $m[C] + d_i > m[C]$). En parcourant j de C vers 0, toute écriture se fait à un indice $j' > j$ déjà traité lors de cette itération : la valeur écrite ne pourra plus servir de source dans la même itération. Chaque dépôt est donc utilisé **au plus une fois**, ce qui est exactement la sémantique voulue (chaque validateur ne peut être recruté qu'une fois).

Contre-exemple en parcours croissant. Prenons $d = [3]$ et $C = 8$. En parcourant j de 0 vers C : $j = 0$ donne $m[3] = 3$; puis, dans la même itération, $j = 3$ (dont la valeur vient d'être écrite) donne $m[6] = 6$; puis $j = 6$ donne $m[8] = 9$. L'algorithme renverrait 9 en utilisant *trois fois* le dépôt 3, alors que la réponse correcte est $+\infty$ (un seul dépôt de 3 ne peut pas atteindre 8).

Question 50. Invariant.

À la fin de la i -ème itération de la boucle externe ($1 \leq i \leq n$), pour tout $0 \leq j \leq C$: $m[j] = m_i(j)$, c'est-à-dire la plus petite somme réalisable à l'aide d'un sous-ensemble de $\{0, \dots, i-1\}$ dont la somme écriée à C vaut exactement j ($+\infty$ si aucun).

Initialisation. Avant la première itération, $m = [0, +\infty, \dots, +\infty]$, c'est bien m_0 .

Hérédité. Supposons $m = m_i$ au début de l'itération $i+1$ (indice i dans le code). Notons m^{fin} le tableau à la fin de cette itération.

($\leq$) Toute valeur écrite dans $m^{\text{fin}}[j']$ est de la forme $m_i(j) + d_i$ avec $\pi(j + d_i) = j'$, donc réalisable par un sous-ensemble de $\{0, \dots, i-1\}$ auquel on ajoute i (l'indice i n'était pas dans ce sous-ensemble par hypothèse de récurrence). Les valeurs conservées sont les $m_i(j)$, réalisables sans i . Donc $m^{\text{fin}}[j'] \geq m_{i+1}(j')$ ne peut être violé : toute valeur du tableau est réalisable, d'où $m^{\text{fin}}[j'] \geq m_{i+1}(j')$.

($\geq$) Réciproquement, soit $S \subseteq \{0, \dots, i\}$ réalisant $m_{i+1}(j')$. Si $i \notin S$, alors $S \subseteq \{0, \dots, i-1\}$ et $m_i(j') \leq \sum_{k \in S} d_k$; comme la case j' n'est jamais augmentée, $m^{\text{fin}}[j'] \leq m_i(j') \leq \sum_{k \in S} d_k$. Si $i \in S$, posons $S' = S \setminus \{i\}$ et $j = \pi(\sum_{k \in S'} d_k)$. Alors $m_i(j) \leq \sum_{k \in S'} d_k$, et l'itération considère le couple (j, j'') avec $j'' = \pi(j + d_i)$. Deux cas : si $\sum_{k \in S'} d_k < C$ alors $j = \sum_{k \in S'} d_k$ et $j'' = \pi(\sum_{k \in S} d_k) = j'$; si $\sum_{k \in S'} d_k \geq C$ alors $j = C$ et $j'' = C = j'$. Dans les deux cas la mise à jour porte sur la case j' avec la valeur $m_i(j) + d_i \leq \sum_{k \in S} d_k$, d'où $m^{\text{fin}}[j'] \leq \sum_{k \in S} d_k = m_{i+1}(j')$.

Enfin, le parcours décroissant garantit (voir question 49) qu'aucune case écrite au cours de cette itération n'est réutilisée comme source, donc l'indice i n'est jamais compté deux fois : les valeurs produites sont bien réalisables sans répétition. L'invariant est établi. $\square$

Conclusion. À la fin, $m[C] = m_n(C) = \mu(d, C)$ d'après la question 48.

Terminaison. Les deux boucles sont des boucles **for** sur des intervalles finis : la terminaison est immédiate. Le nombre total d'itérations est exactement $n(C+1)$.

Question 51. Complexité. Temps : $\Theta(nC)$ (deux boucles imbriquées, corps en $O(1)$). Espace : $\Theta(C)$ auxiliaire (un unique tableau de taille $C+1$), contre $\Theta(nC)$ pour une version conservant tous les m_i .

Pseudo-polynomialité. La taille de l'entrée n'est pas C mais le nombre de *bits* nécessaires pour l'écrire, soit $\Theta(\log C)$. La complexité $\Theta(nC) = \Theta(n 2^{\log_2 C})$ est donc *exponentielle* en la taille de l'entrée : on parle d'algorithme **pseudo-polynomial**, polynomial en la *valeur* des données mais non en leur *taille*. Il n'y a donc aucune contradiction avec la NP-complétude établie à la question 46.

Passage aux wei. Si les dépôts sont exprimés en wei, C est multiplié par 10^{18} . La complexité devient de l'ordre de $n \times 10^{18} \times$ (seuil en ether) : le calcul est totalement irréalisable, et le tableau ne tiendrait pas en mémoire. En pratique on change d'unité (par exemple le *gwei*, ou mieux le PGCD des dépôts) ou l'on renonce à l'optimum exact au profit d'une heuristique gloutonne (trier les dépôts par ordre décroissant et recruter jusqu'à atteindre C), qui donne ici une 2-approximation.

Question 52. Avec $d = [12, 7, 20, 5, 16]$:

$$T = 12 + 7 + 20 + 5 + 16 = 60, \quad C = \left\lceil \frac{2 \times 60}{3} \right\rceil = 40.$$

On trouve $\mu(d, 40) = 40$, atteint par le sous-ensemble

$$S^* = \{12, 7, 5, 16\}, \quad 12 + 7 + 5 + 16 = 40.$$

(C'est le seul sous-ensemble de somme exactement 40.) L'attaquant doit donc contrôler quatre validateurs sur cinq, mais peut se dispenser du plus gros dépôt.

Déroulé du tableau pour la version réduite $d = [3, 5, 7]$ et $C = 8$ (on note ∞ pour $+\infty$) :

j	0	1	2	3	4	5	6	7	8
initialisation	0	∞	∞	∞	∞	∞	∞	∞	∞
après $d_0 = 3$	0	∞	∞	3	∞	∞	∞	∞	∞
après $d_1 = 5$	0	∞	∞	3	∞	5	∞	∞	8
après $d_2 = 7$	0	∞	∞	3	∞	5	∞	7	8

D'où $\mu([3, 5, 7], 8) = m_3(8) = 8$, réalisé par $\{3, 5\}$. On vérifie au passage la remarque de la question 47 : pour $j < 8$, chaque case vaut j ou ∞ ; seule la case $j = 8$ agrège plusieurs sommes (8, 10, 12, 15) et n'en garde que le minimum 8. Noter aussi que la case $j = 8$ passe de ∞ à 8 dès le traitement de $d_1 = 5$ (via $3 + 5$) et n'est plus améliorée ensuite.

Question 53. Sur l'exemple, $\mu(d, C) = 40 = C$: l'attaquant atteint *exactement* le seuil.

Ce n'est pas toujours possible. Prenons $d = [1, 100]$. Alors $T = 101$ et $C = \lceil 202/3 \rceil = 68$. Les sommes réalisables sont 0, 1, 100 et 101 ; les seules ≥ 68 sont 100 et 101, donc

$$\mu(d, C) = 100 > 68 = C.$$

La granularité des dépôts empêche d'atteindre le seuil : pour réunir une supermajorité, l'attaquant est contraint de mobiliser 100, soit près de 99 % du dépôt total, alors que 67,3 % auraient « suffi » en théorie. Un système où quelques validateurs concentrent l'essentiel du dépôt rend donc l'attaque *plus chère en valeur absolue*, mais aussi beaucoup plus facile à monter en pratique (il suffit de corrompre un seul acteur) : la métrique pertinente pour la sécurité réelle est le nombre d'acteurs à corrompre autant que le capital à mobiliser.

Lien avec la question 7. Il faut distinguer deux quantités :

- le dépôt *mobilisé* pour former une supermajorité, au moins $C = \lceil 2T/3 \rceil$ (ici 40, soit $\frac{2}{3}T$) ;
- le dépôt effectivement *détruit* lors d'une violation de la sûreté, au moins $T/3$ (ici 20) d'après le lemme d'intersection.

Un attaquant qui contrôle $2T/3$ et finalise deux points de contrôle en conflit perd donc au minimum la moitié de ce qu'il a mobilisé. C'est ce ratio, et non la seule difficulté algorithmique, qui constitue la véritable barrière économique de Casper.

Question 54. Synthèse. Les parties III et IV établissent la *sûreté responsable* : sous l'hypothèse que moins d'un tiers du dépôt triche, deux points de contrôle en conflit ne peuvent être finalisés, et si l'hypothèse tombe, les coupables sont identifiables et punis (partie IV, question 40). Cette garantie est *inconditionnelle vis-à-vis du réseau* : elle ne suppose aucune borne sur la latence.

En revanche elle ne dit rien de la *vivacité*. Si plus d'un tiers des validateurs cessent de voter — panne massive, partition réseau, censure — plus aucune supermajorité ne peut se former : aucun lien de supermajorité n'apparaît, l'ensemble J de la question 26 cesse de croître, et la chaîne ne finalise plus rien. Le protocole est bloqué sans que personne n'ait triché.

C'est exactement le rôle de la **fuite d'inactivité** de la partie I : en faisant décroître géométriquement le dépôt des absents (question 8), elle diminue le dépôt total T et donc le seuil $C = \lceil 2T/3 \rceil$,

jusqu'à ce que les validateurs restés actifs redeviennent, à eux seuls, une supermajorité. Le nombre d'époques nécessaires est le k^* de la question 9, logarithmique en le rapport des dépôts : la reprise est relativement rapide. La vivacité est ainsi restaurée.

Mais ce mécanisme a un prix. La démonstration de la question 40 repose sur le fait que les deux supermajorités en cause sont mesurées *avec le même* dépôt total T ; c'est ce qui autorise le lemme d'intersection. Si le réseau se scinde en deux partitions $\mathcal{V}_A$ et $\mathcal{V}_B$, chacune voit l'autre comme inactive et lui applique la fuite : sur la chaîne A le dépôt de $\mathcal{V}_B$ fond, et réciproquement. Au bout d'un certain temps, $\mathcal{V}_A$ est une supermajorité *selon les comptes de la chaîne A* et $\mathcal{V}_B$ une supermajorité *selon ceux de la chaîne B* : deux points de contrôle en conflit sont finalisés sans qu'aucun validateur n'ait enfreint le moindre commandement, donc **sans qu'aucun dépôt ne soit détruit**. L'hypothèse du lemme d'intersection est violée, non par malveillance, mais parce que les deux supermajorités ne se rapportent plus au même T .

Casper échange donc une garantie de sûreté *absolue* contre une garantie de sûreté *sous hypothèse de synchronie faible*, assortie d'une garantie de vivacité. C'est un compromis classique et inévitable : le théorème CAP, comme l'impossibilité FLP, interdit d'obtenir simultanément la sûreté et la vivacité dans un réseau asynchrone sujet aux partitions. Le protocole prescrit alors une règle simple et purement locale : *chaque validateur retient le premier point de contrôle finalisé qu'il a vu*, et l'arbitrage entre les deux branches est renvoyé hors du protocole, à un embranchement logiciel décidé par la communauté.